Task 7: Configure Azure Load Balancer for Two Linux Virtual Machines

Objective

To deploy web applications on two Ubuntu Linux Virtual Machines using Nginx and configure an Azure Standard Public Load Balancer to distribute incoming traffic across both backend virtual machines, ensuring high availability and efficient traffic distribution.

Resource Used

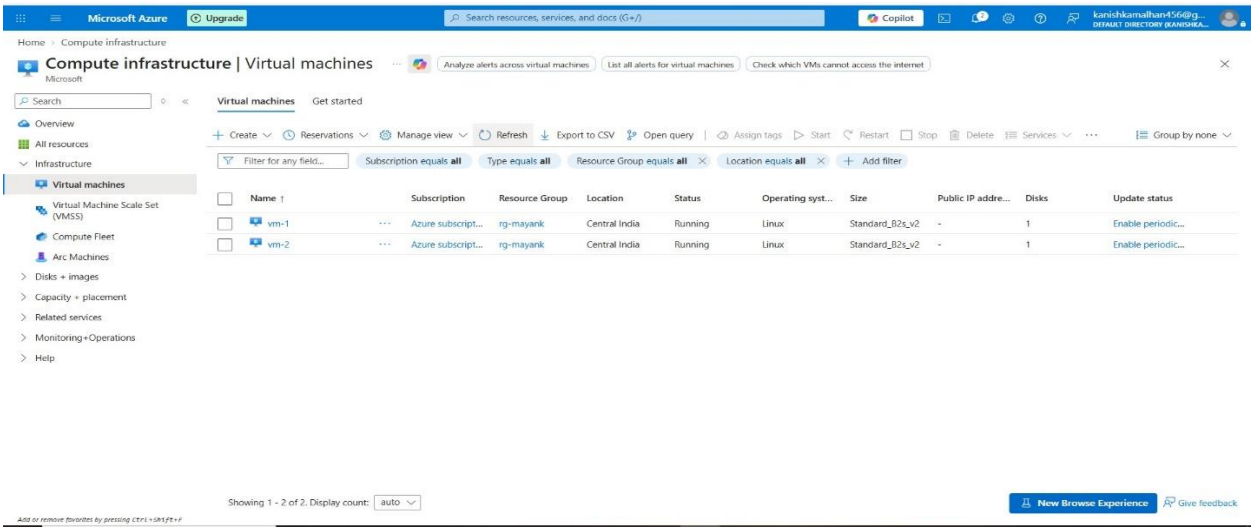
- Azure Subscription
- Resource Group: **rg-mayank**
- Two Ubuntu Linux Virtual Machines (VM-1 and VM-2)
- Virtual Network (VNet)
- Nginx Web Server
- GitHub Repository containing web application files
- Network Security Group (NSG) with Port 80 allowed

Step 1: Verify Virtual Machines

Verified that both Ubuntu Linux virtual machines were successfully deployed and running.

Observation

- VM-1 and VM-2 were in running state.
- Both virtual machines were deployed in the Central India region.
- Both servers were prepared to act as backend servers for the Load Balancer.



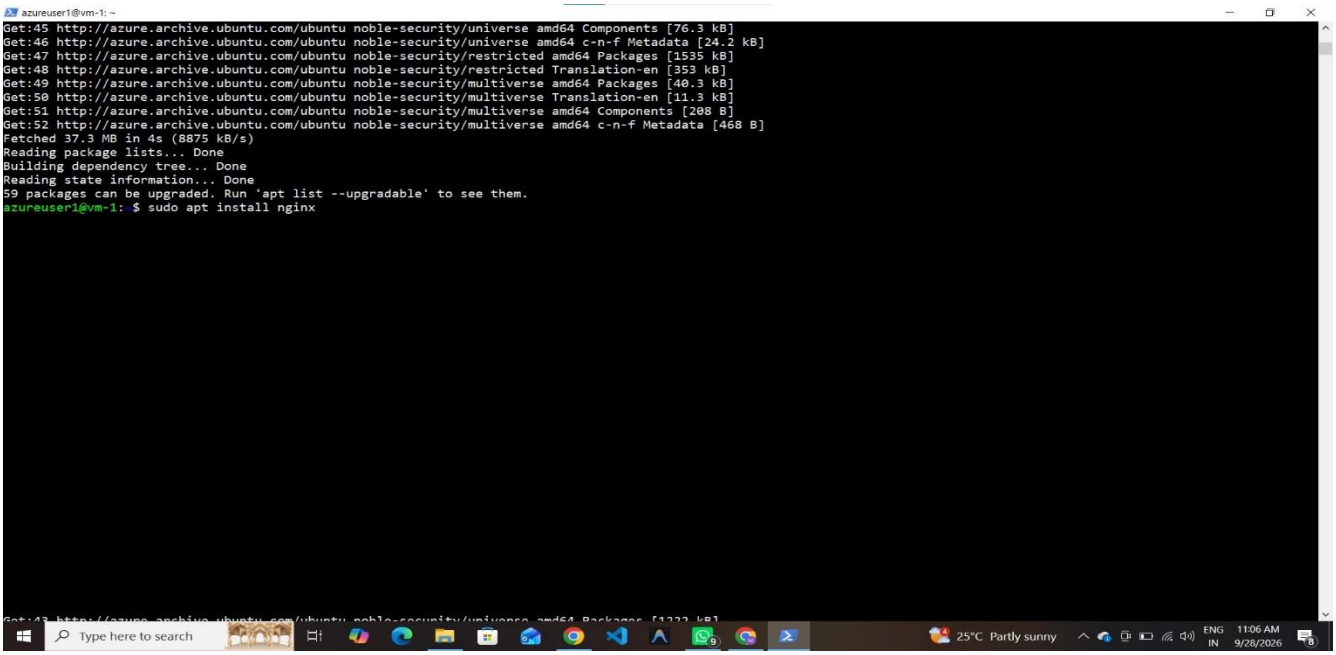
Step 2: Install Nginx on VM-1

Connected to VM-1 through SSH and installed the Nginx web server.

sudo apt update
sudo apt install nginx -y

Observation

- Package repository was updated successfully.
- Nginx web server was installed successfully.
- VM-1 became ready for web application hosting.



Step 3: Verify Default Nginx Web Directory

Navigated to the default Nginx web directory.

cd /var/www/html
ls

Observation

- Default Nginx landing page was present.
- The Nginx web root directory was identified successfully.



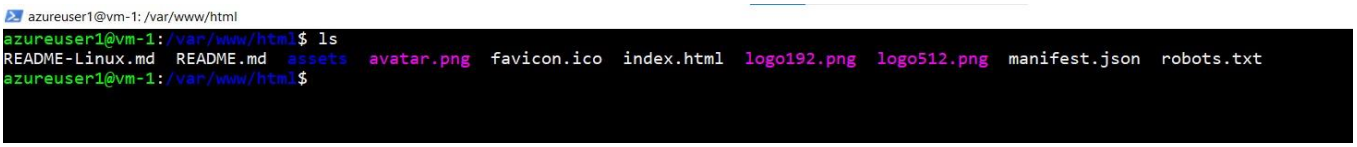
Step 4: Deploy Application on VM-1

Removed the default Nginx page and copied the application files obtained from GitHub into the Nginx web root directory.

sudo rm -f /var/www/html/index.nginx-debian.html
sudo cp -r * /var/www/html/

Observation

- Default Nginx page was removed successfully.
- Application files were copied to the web root directory.
- VM-1 was prepared to host the web application.



Step 5: Verify Application Files on VM-1

Verified the presence of application files in the Nginx web root directory.

ls

Observation

- All required application files were available.
- The web application was successfully deployed on VM-1.
- Files such as `index.html`, `assets`, and supporting resources were present.

Step 6: Deploy Application on VM-2

Repeated the same deployment process on VM-2.

Observation

- Application files were successfully deployed on VM-2.
- Both backend servers were hosting web applications through Nginx.

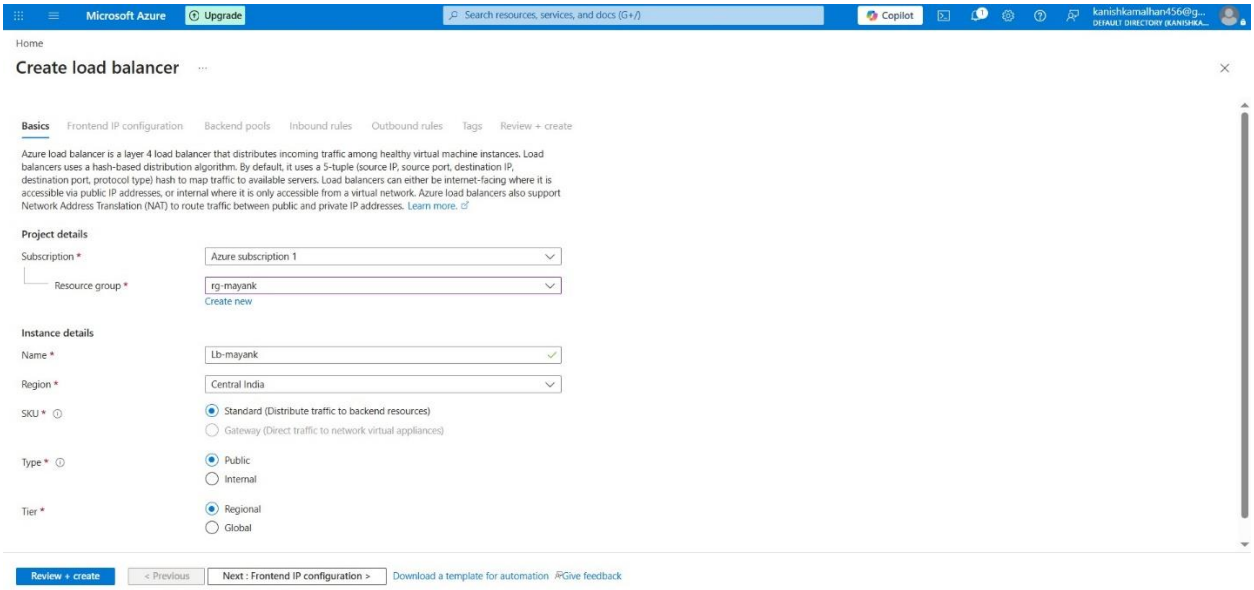


Step 7: Create Azure Load Balancer

Created an Azure Standard Public Load Balancer.

Observation

- Azure Standard Public Load Balancer was created successfully.
- The Load Balancer was configured to receive internet traffic.

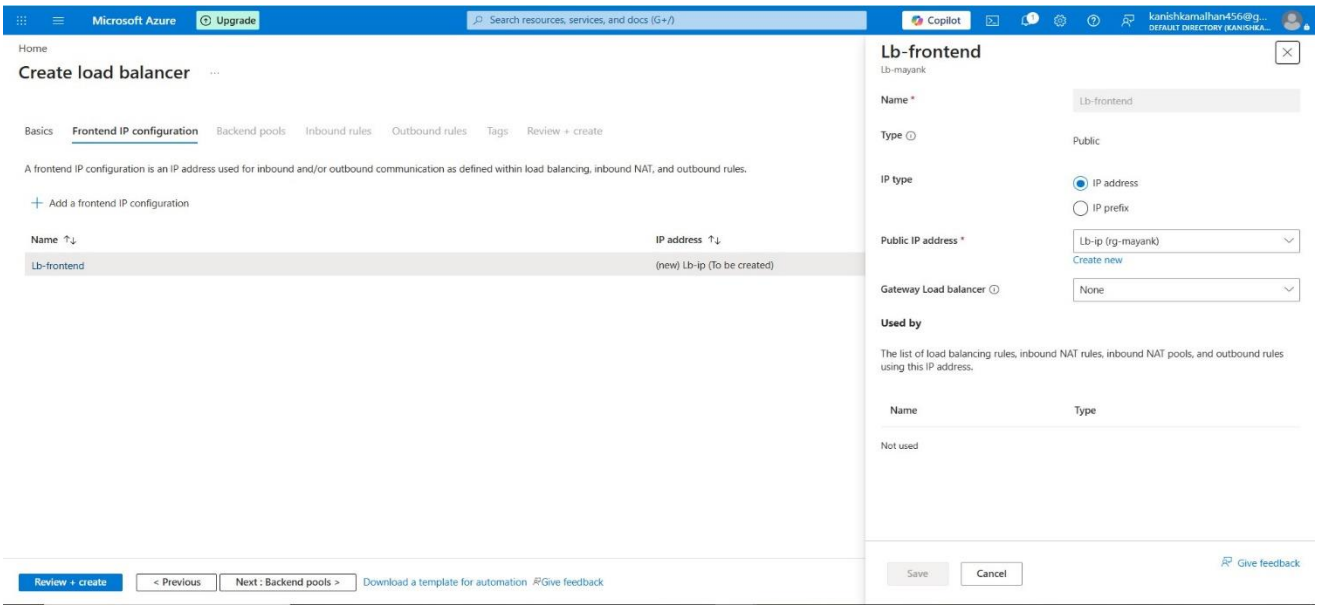


Step 8: Configure Frontend IP Address

Created a frontend IP configuration using a Public IP Address.

Observation

- Public frontend endpoint was configured successfully.
- Internet traffic could now reach the Load Balancer through the assigned Public IP.

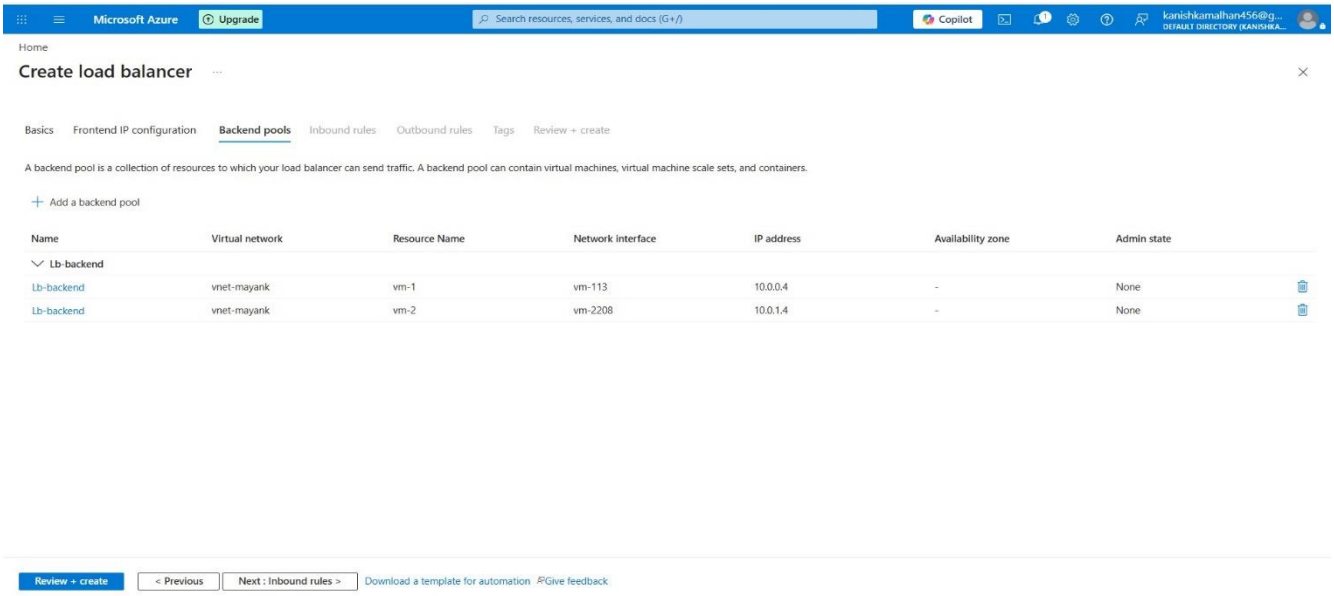


Step 9: Configure Backend Pool

Created a backend pool and added both virtual machines as backend resources.

Observation

- Backend Pool was created successfully.
- VM-1 and VM-2 were added as backend instances.
- The Load Balancer could distribute traffic across both virtual machines.

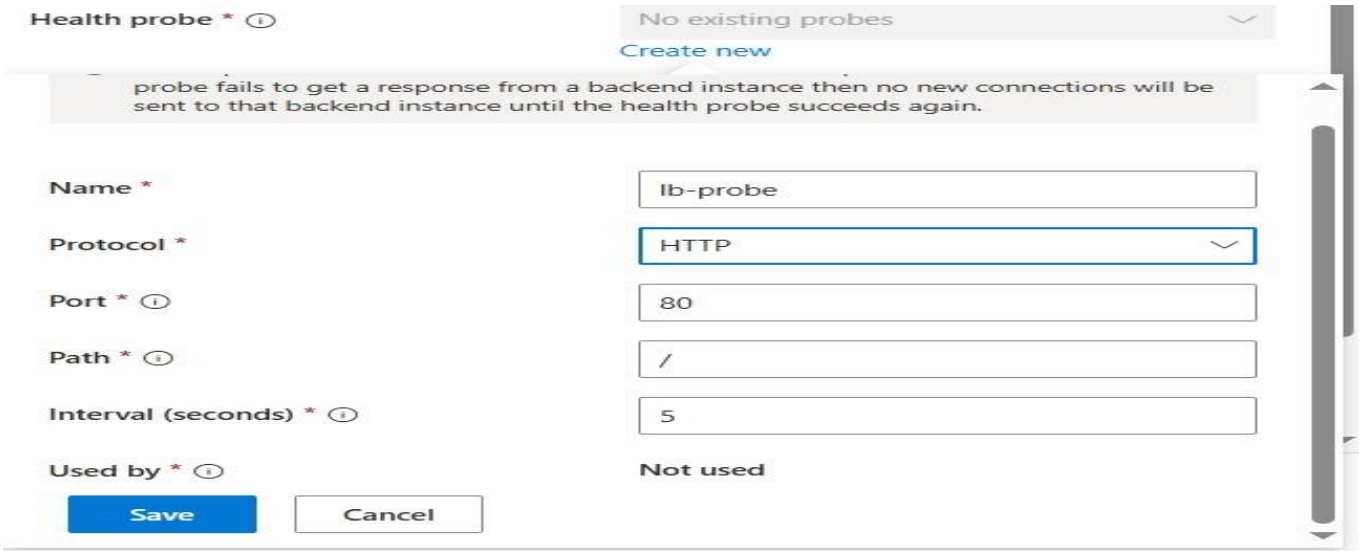


Step 10: Configure Health Probe

Created an HTTP Health Probe to monitor backend server availability.

Observation

- Health Probe was configured successfully.
- Backend server availability was continuously monitored.
- Only healthy backend servers could receive traffic.

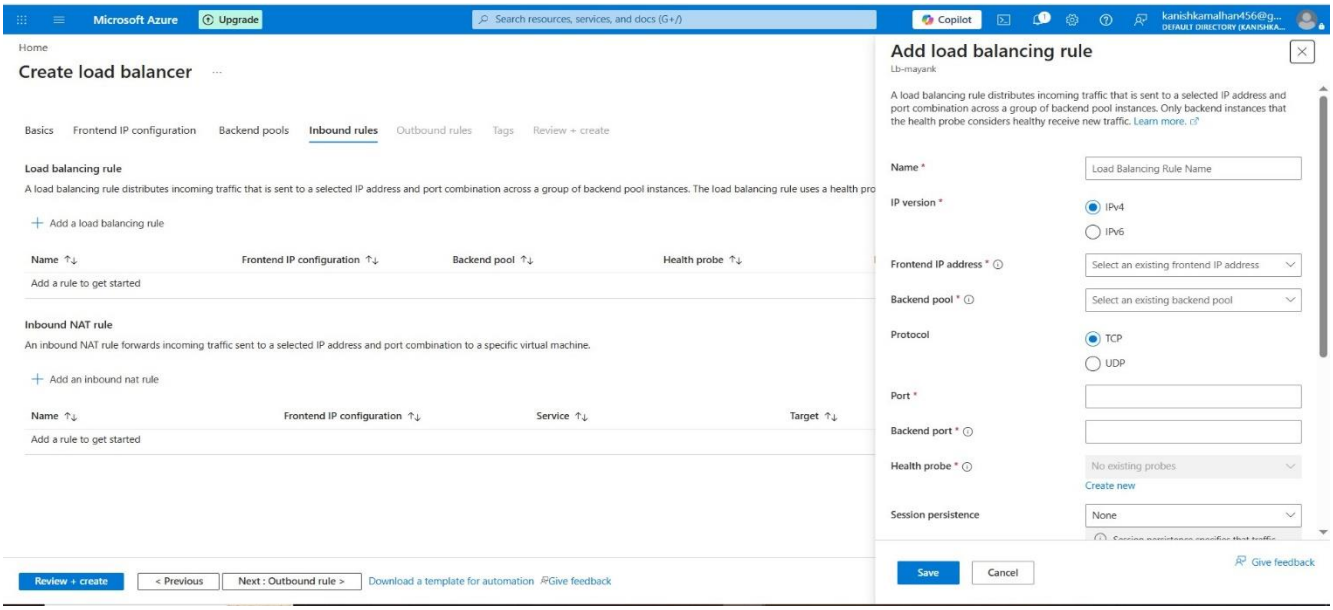


Step 11: Configure Load Balancing Rule

Created a Load Balancing Rule to forward traffic from the frontend IP to backend servers.

Observation

- Load Balancing Rule was configured successfully.
- Frontend traffic on Port 80 was mapped to backend instances.
- Health Probe was integrated with the Load Balancing Rule.

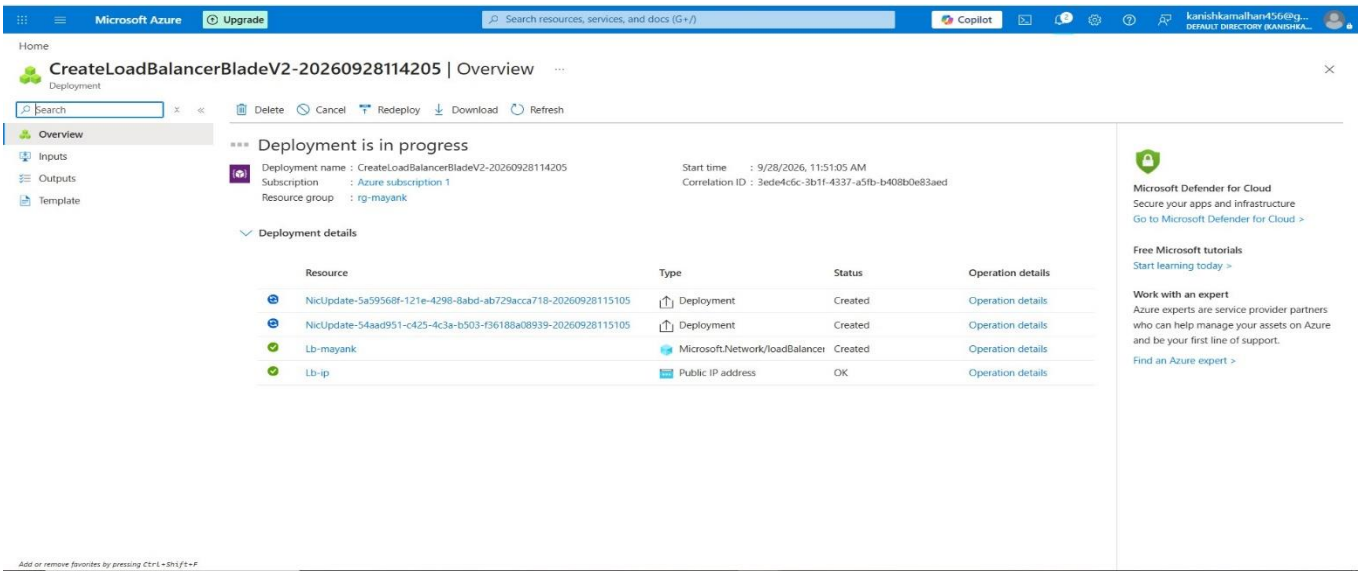


Step 12: Review Deployment Status

Reviewed the deployment status after configuration.

Observation

- Load Balancer deployment completed successfully.
- Public IP resource was provisioned successfully.
- Backend resources were associated without any errors.

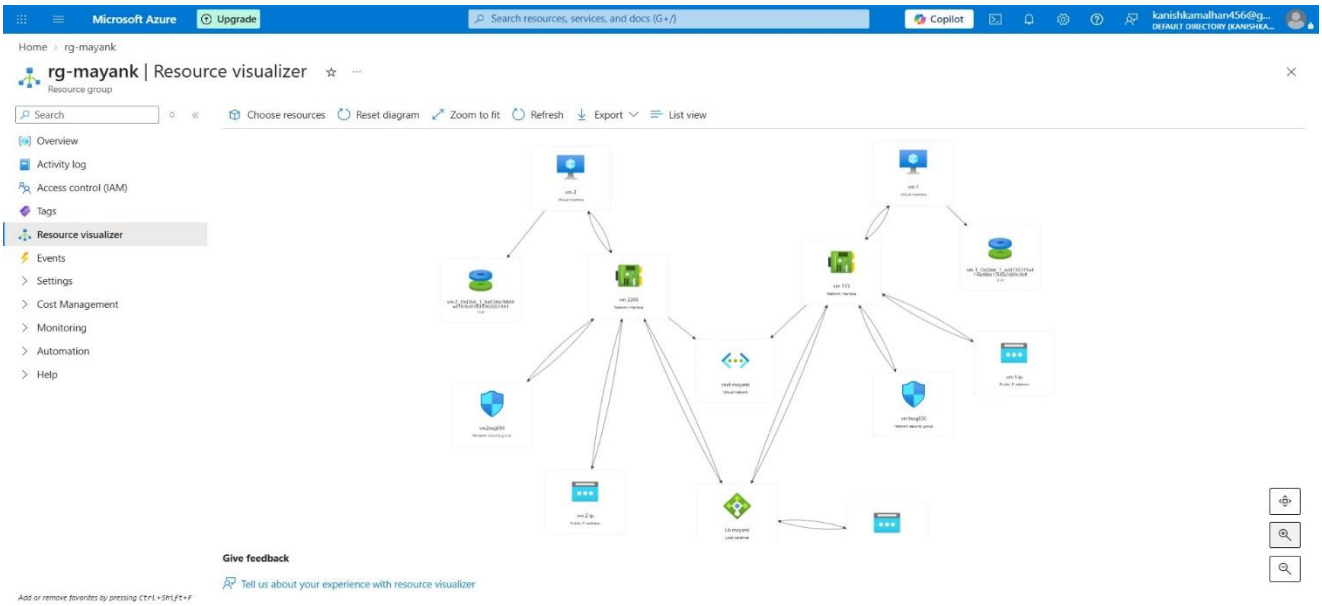


Step 13: Validate Resource Connectivity

Used Azure Resource Visualizer to verify resource connectivity and architecture.

Observation

- Both virtual machines were connected to the virtual network.
- Load Balancer was associated with backend resources.
- Public IP was connected successfully to the frontend configuration.
- Resource relationships were verified successfully.

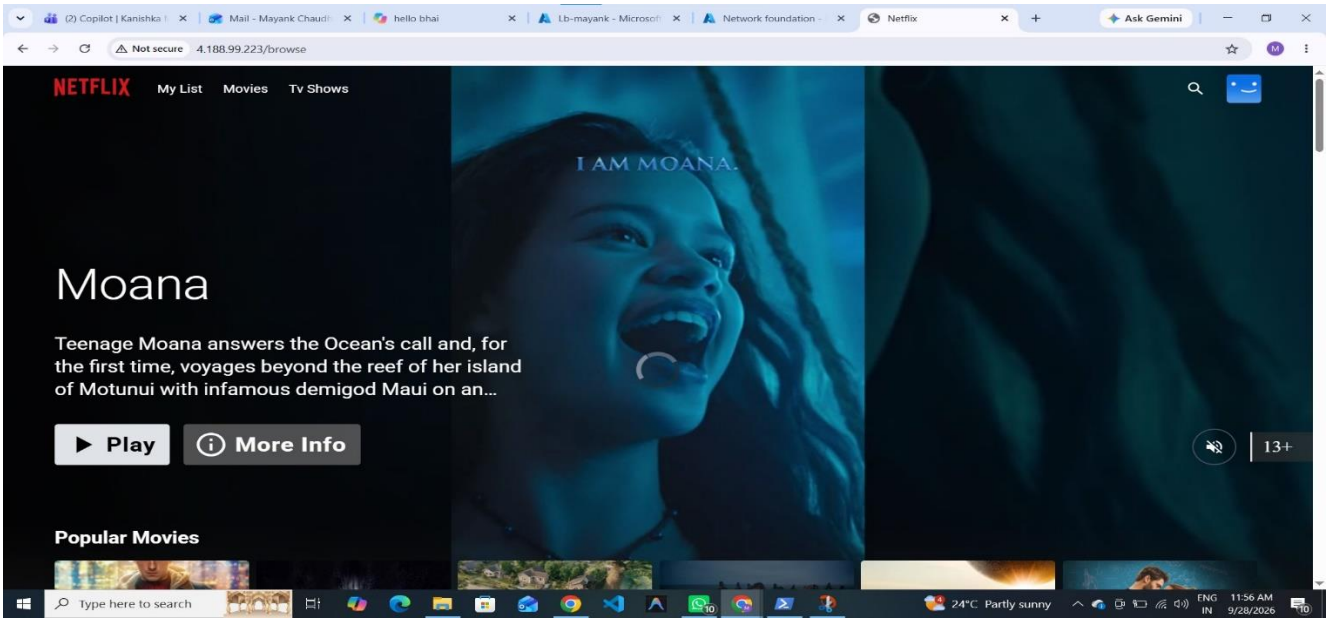


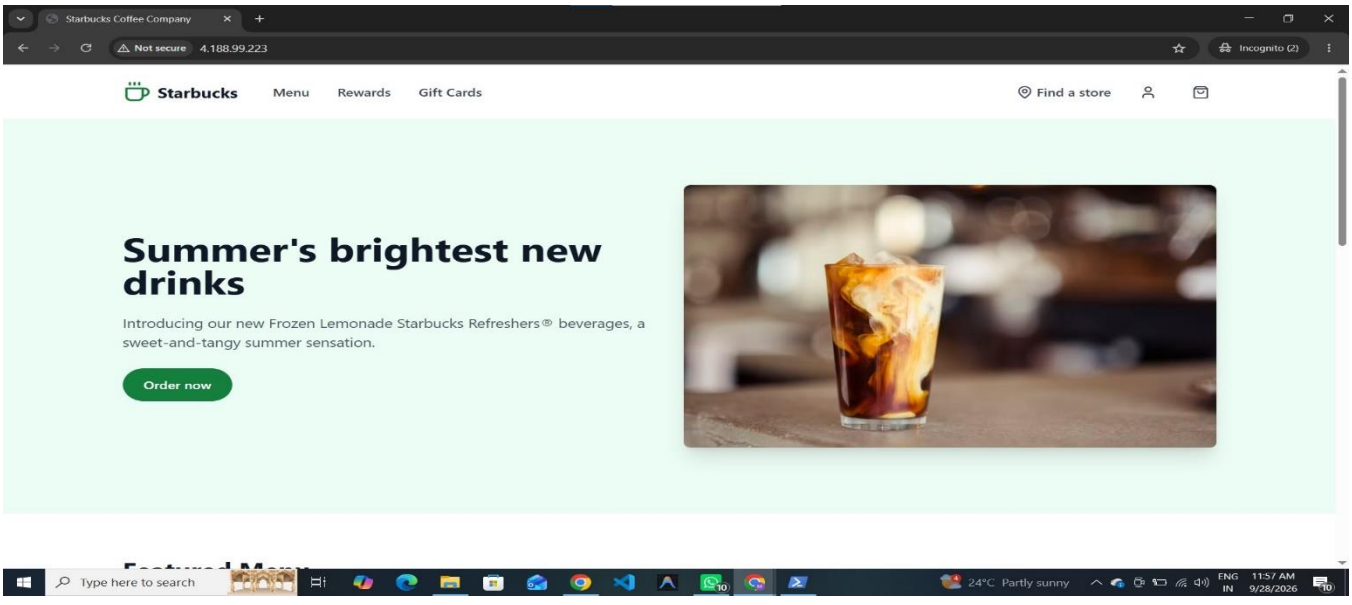
Step 14: Verify Load Balancer Functionality

Accessed the application using the Load Balancer Public IP address and verified successful traffic distribution.

Observation

- Web applications were accessible through the Load Balancer Public IP.
- Requests were routed successfully to backend instances.
- Different application pages hosted on VM-1 and VM-2 were served successfully.
- Load Balancer distributed client requests across both virtual machines.





Result

Successfully deployed web applications on two Ubuntu Linux Virtual Machines using Nginx and configured an Azure Standard Public Load Balancer. The backend pool contained both virtual machines, an HTTP Health Probe monitored server health, and incoming traffic was successfully distributed across both backend servers using a single Public IP address.

Final Observation

- Two Ubuntu Linux Virtual Machines were configured successfully.
- Nginx Web Server was installed on both virtual machines.
- GitHub-based web applications were deployed successfully.
- Azure Standard Public Load Balancer was created successfully.
- Frontend Public IP was configured.
- Backend Pool was configured with VM-1 and VM-2.
- HTTP Health Probe was configured on Port 80.
- Load Balancing Rule was created successfully.
- Deployment completed without errors.
- Resource connectivity was verified using Resource Visualizer.
- Applications were accessible through the Load Balancer Public IP.
- Traffic was successfully distributed across both backend virtual machines.
- High availability and load distribution objectives were achieved successfully.

Final Result

Azure Standard Public Load Balancer was successfully configured, and incoming client requests were distributed between VM-1 and VM-2, ensuring improved availability, scalability, and fault tolerance for the hosted web applications.